# Падающие тени

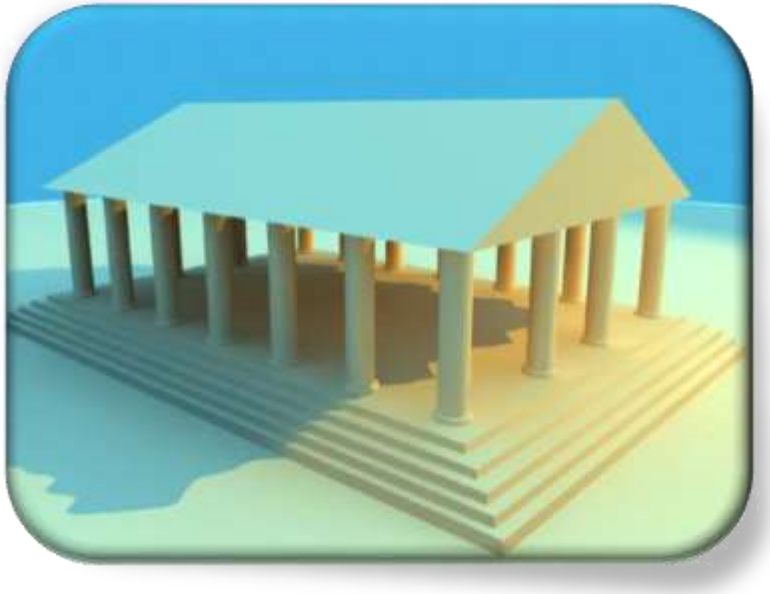
# Почему тени так важны?

# Shadowing Techniques

Применение теней к визуализированной сцене повышает реализм и дает визуальную подсказку о глубине и положении объектов

Два основных вида теней в играх:

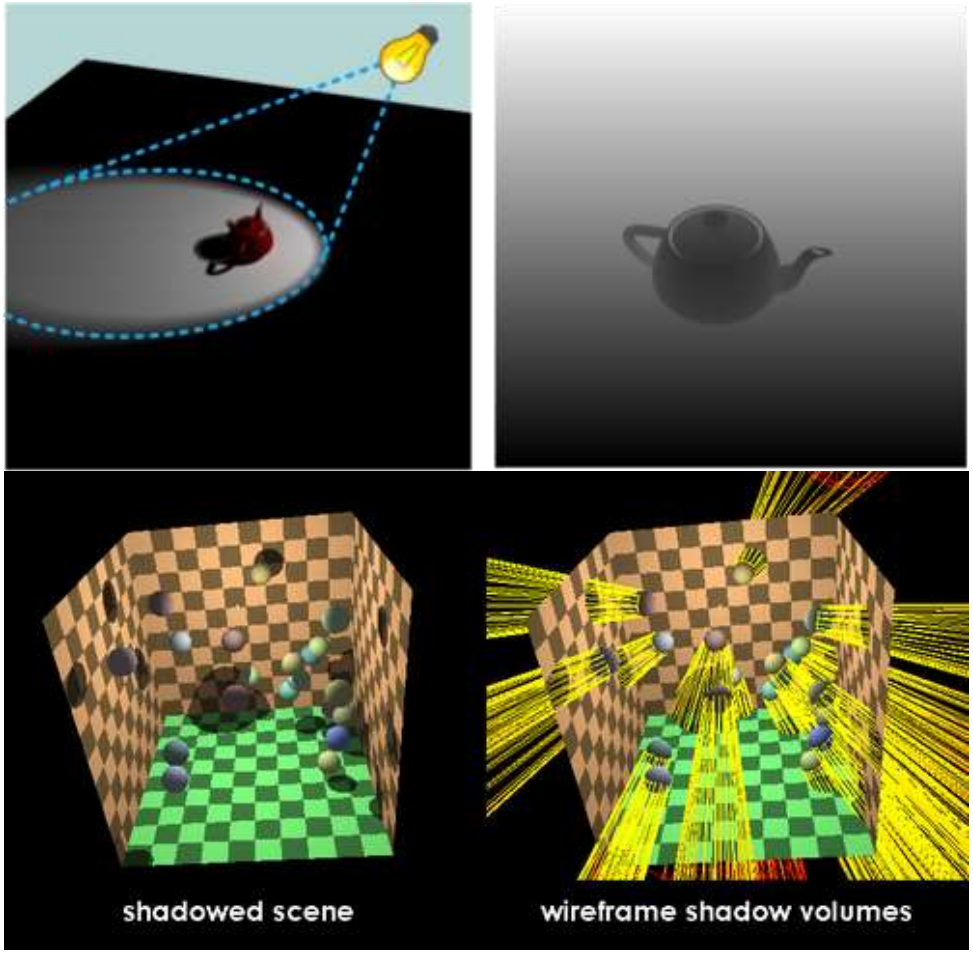
- Жёсткие (Hard)
- Мягкие (Soft)



# Shadowing Techniques

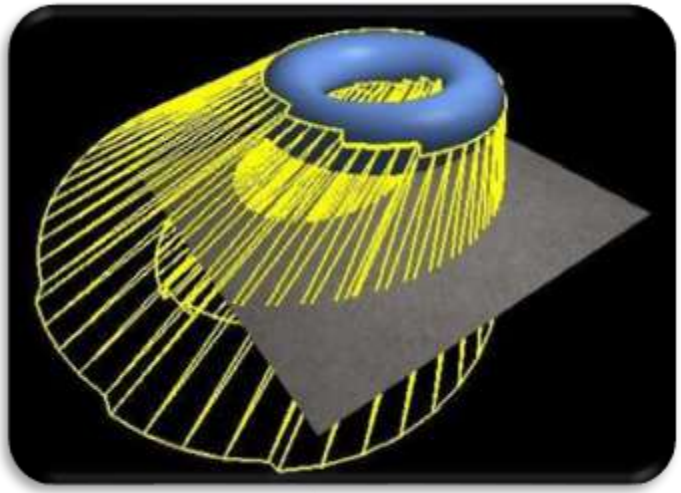
Два наиболее распространенных подхода к созданию жестких теней в играх:

- Карты теней (Shadow maps)
- Объёмы теней (Shadow volumes, Stencil shadows)



# Shadow Volumes

- Геометрический подход
- Геометрия «выдавливается» в направлении от источника света для создания замкнутого объема тени
- Путем тестирования по объему тени можно определить затененные части сцены (Буфер трафарета может быть использован для определения освещенной области)



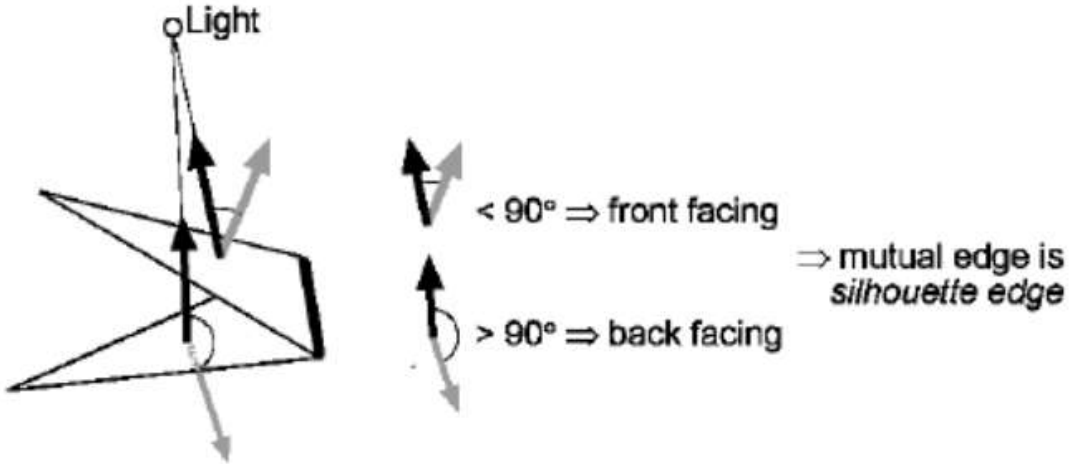
- Объемы теней имеют пиксельную точность и не страдают от проблем с «лесенкой»
- Вычислительные затраты зависят от геометрии (карты теней часто быстрее)

D3D9 405.42 fps Vsync off (760x563), X8R8G8B8 (D24S8)
HAL (hw vp): NVIDIA GeForce 8800M GTX
Toggle full screen
Toggle REF (F3)
Change device (F2)
Background: Field
1 light
Show shadow volume
Luminance: 15.0
Change mesh
Controls (F1 to hide):
Camera control: Left mouse
Mesh control: Right mouse
Light control: Middle mouse
Quit: ESC
W/S/A/D/Q/E to move camera.

# Shadow Volumes: Creating Shadow Volumes (1)

Определение рёбер силуэта:

- Ребро является ребром силуэта, если оно общее для треугольников, обращенных к камере (front faced) и от камеры (back faced)



# Shadow Volumes: Creating Shadow Volumes (2)

После того, как мы определили рёбра силуэта, нам нужно их выдавить to infinity & beyond от источника света

Дополнительно создайте «колпачки»

w-координата однородных координат может быть установлена равной нулю, чтобы продлить точку до бесконечности
xyz-координаты представляют направление

# Shadow Volumes: Creating Shadow Volumes (3)

- Пиксель находится внутри объемов?
  - Если его нет внутри, то он освещён
  - Если внутри, то не освещён

- Как это сделать:
  - Depth pass
  - Depth fail
  - Exclusive-or

# Shadow Volumes: Stencil Buffer Approach

- Рисуем сцену только с окружающим (ambient) освещением

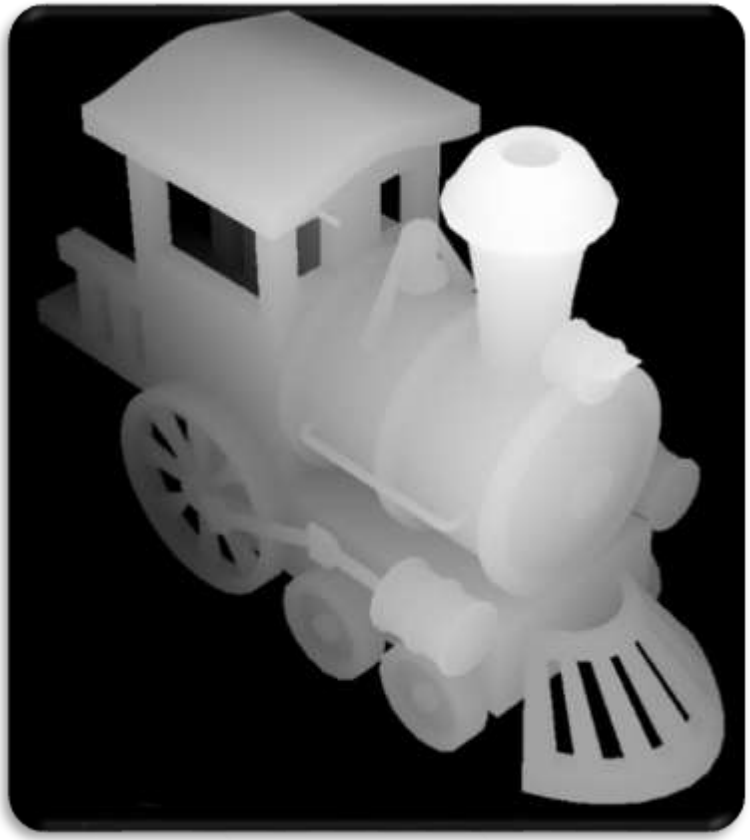
Depth pass:

- Очищаем буфер трафарета
- Отключаем запись в буферы глубины и цвета
- Рисуем объёмы с back-face culling
  - Инкрементируем значение трафарета на depth pass
- Рисуем объёмы с front-face culling
  - Декрементируем значение трафарета на depth pass
- После рендеринга, если значение трафарета равно нулю, он находится за пределами тени, поэтому должен быть освещен
- Считаем оставшиеся компоненты освещения, но только для пикселей, где трафарет равен нулю

# Shadow Maps

Карты теней — это рендеринг сцены с точки зрения света, где хранятся глубины до «освещенных» фрагментов

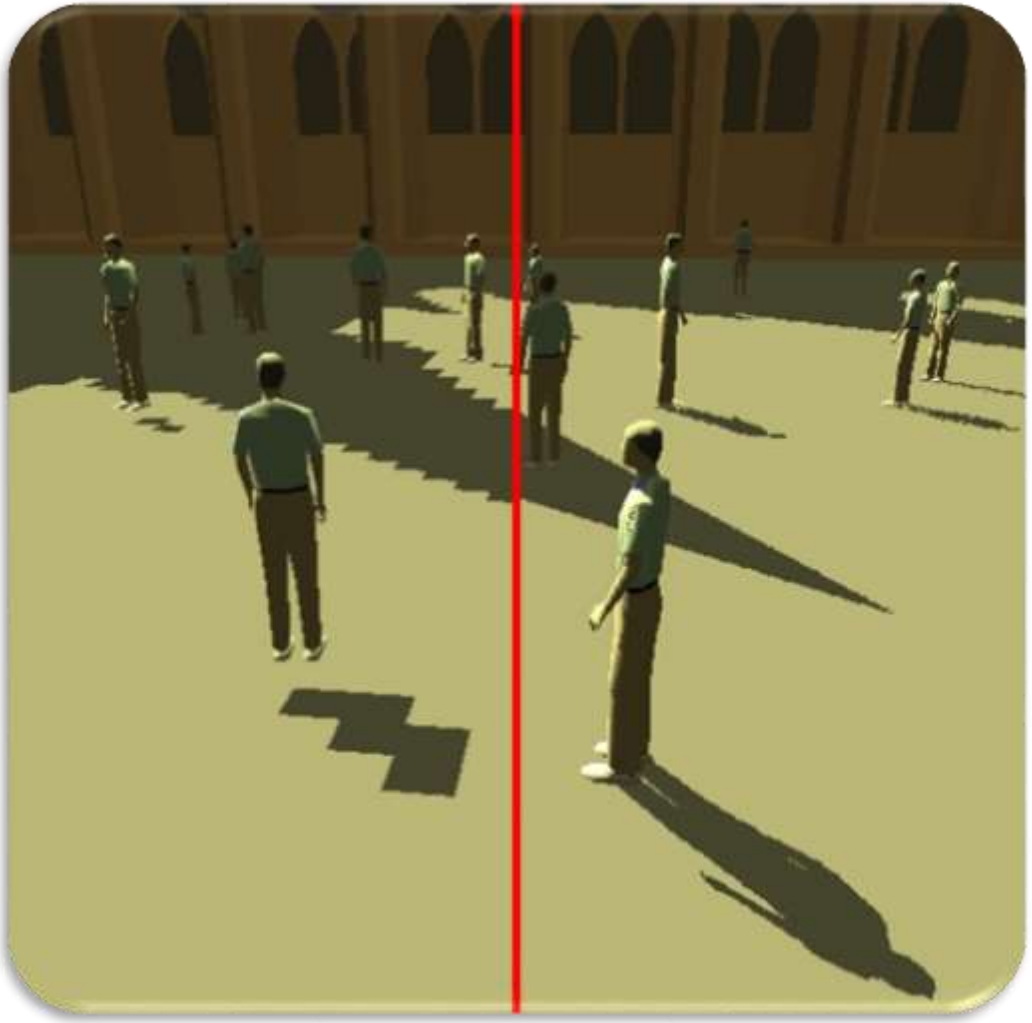
- При рендеринге сцены с точки зрения наблюдателя значения глубины пикселей сравниваются с значениями глубины в карте теней
- Если глубина больше, чем значение, хранящееся в карте теней, то пиксель находится в тени



# Shadow Maps

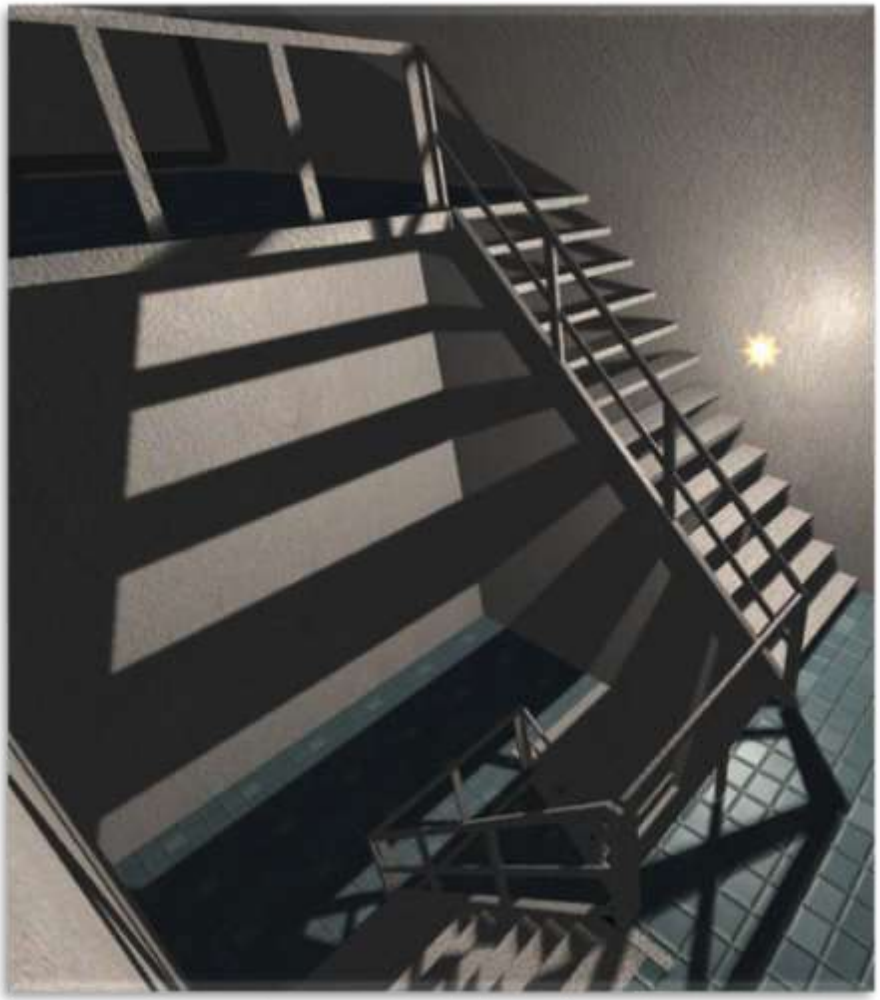
Качество карты теней зависит от размера сгенерированной карты теней

- 256x256
- 1024x1024
- 4096x4096
- 1024x4096



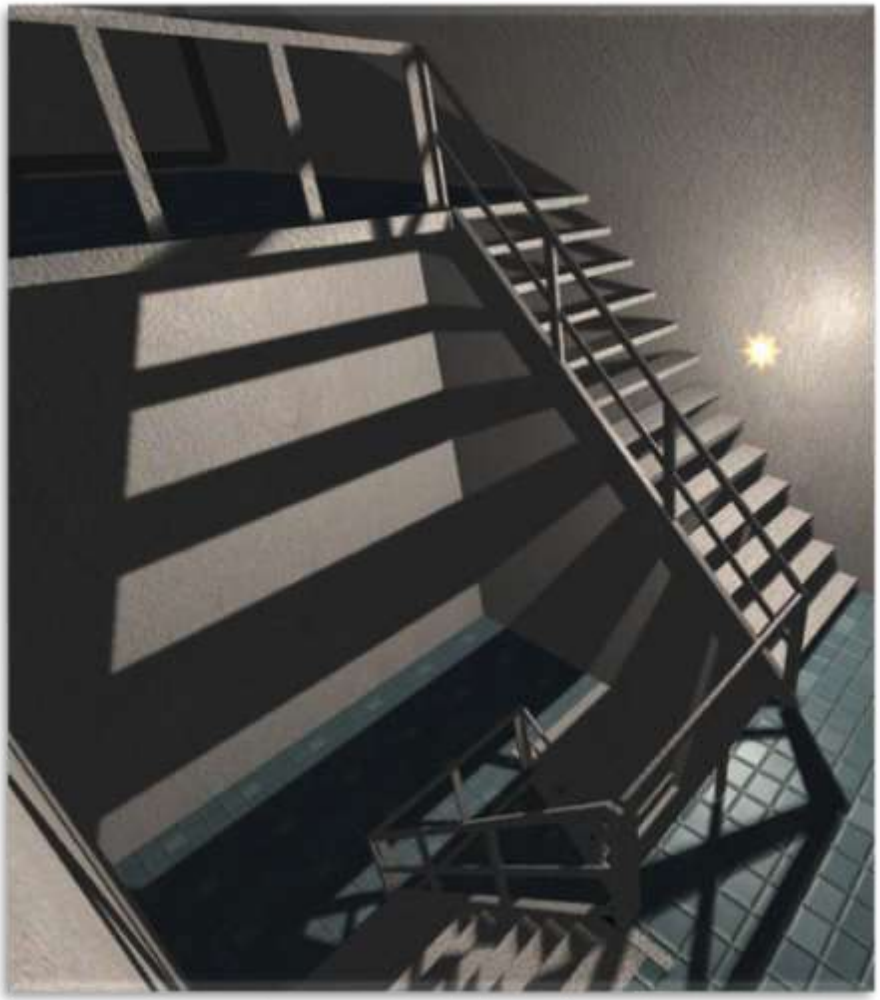
# Shadow Maps: Creating the Shadow Map

- Рендеринг сцены с точки зрения источника света
  - точечный свет
    - перспективная проекция
    - «6» камер в источнике света
  - направленный свет
    - ортогональная проекция
    - позицию камеры надо подобрать так, чтобы сцена полностью попадала во фрустум камеры



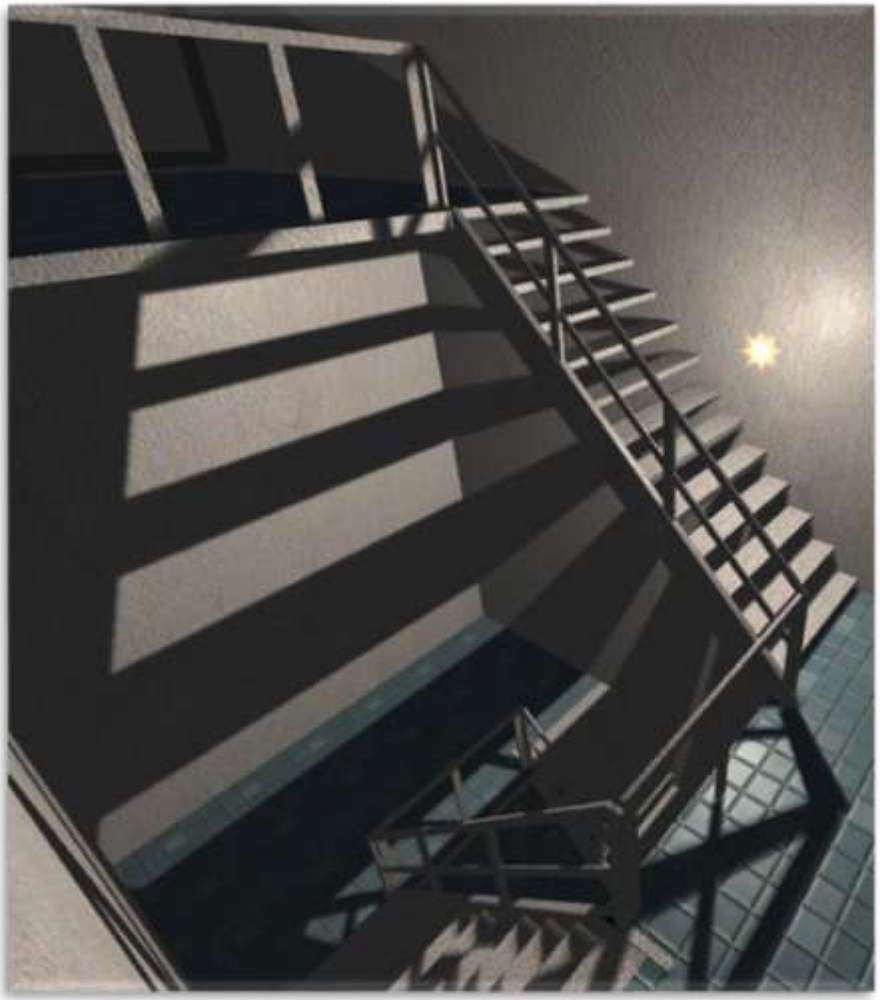
# Shadow Maps: Creating the Shadow Map

- Карта глубины обычно хранится в виде текстуры
- Пиксельный шейдер может быть установлен в null
- Не все объекты нужно рисовать в карте теней, можно выбрать только важные объекты, отбрасывающие тени



# Shadow Maps: Creating the Shadow Map

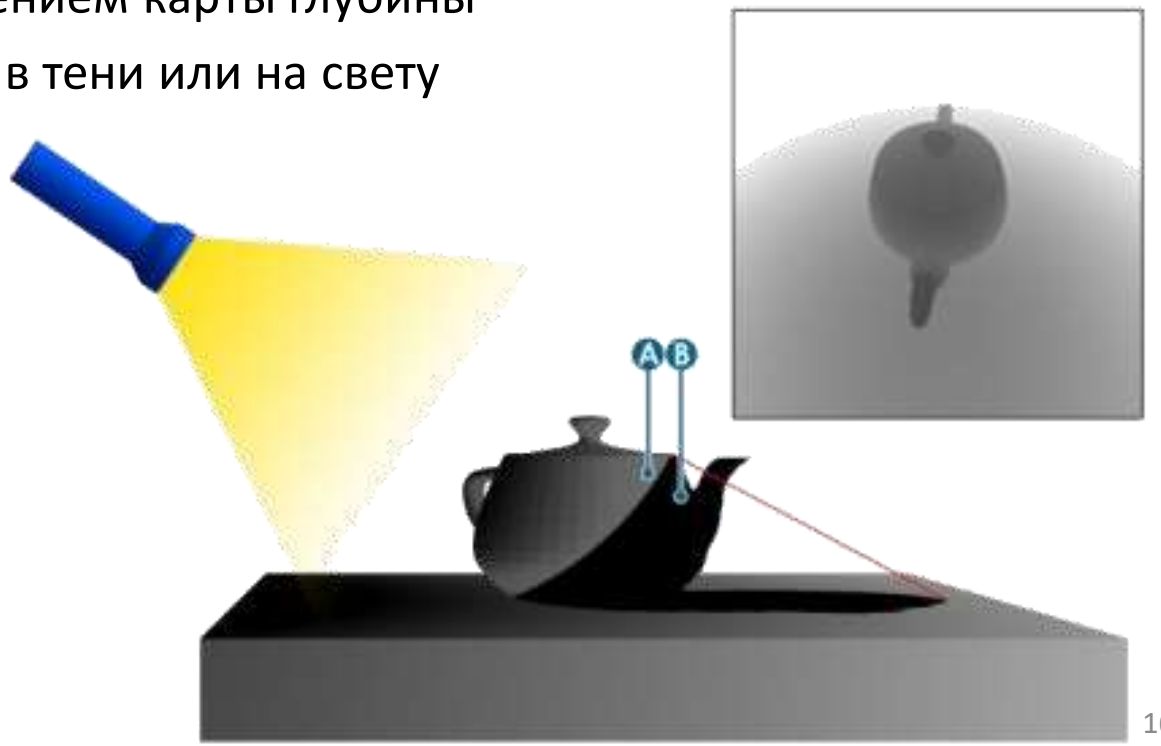
- Если свет не движется и геометрия отбрасывающая тени не перемещается, то карту теней можно не пересчитывать и её можно кэшировать/использовать повторно
- Для каждого источника света отбрасывающего тени нужна своя карта теней
- Смещение глубины, двигающее объекты в сторону от света, обычно применяется для устранения проблем с точностью буфера глубины (где значение карты глубины очень близко к глубине рисуемой поверхности)

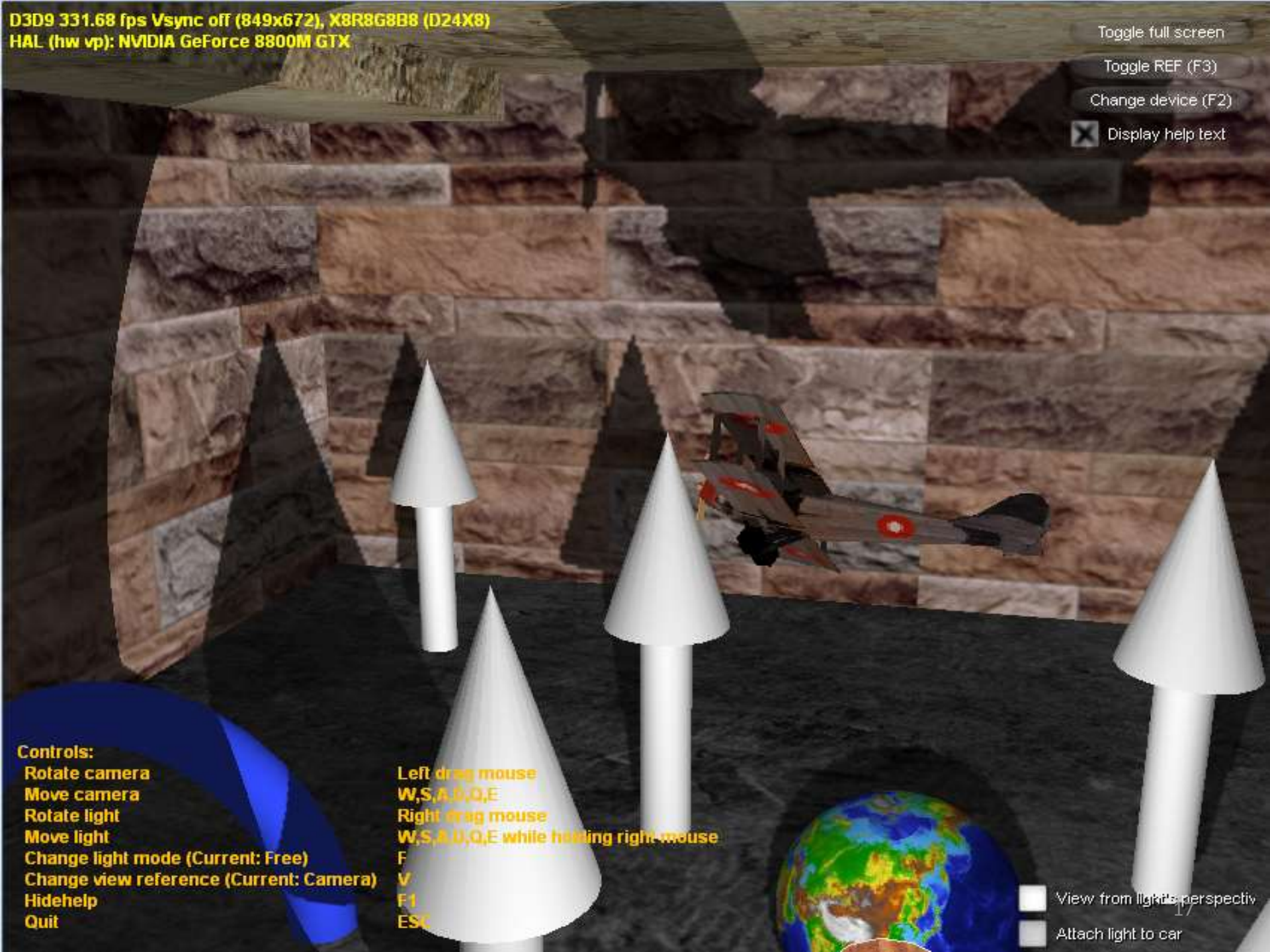


# Shadow Maps: Using the Shadow Map

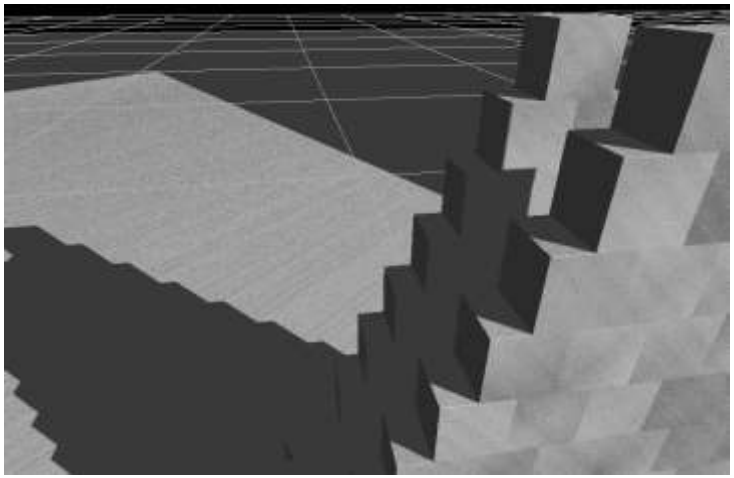
При рисовании сцены с точки зрения камеры карта теней используется для:

- Нахождения положения пикселя с точки зрения источника света
- Сравнения расстояния от положения до источника света с сохраненным значением карты глубины
- Отрисовке пикселя в тени или на свету




D3D9 331.68 fps Vsync off (849x672), X8R8G8B8 (D24X8)
HAL (hw vp): NVIDIA GeForce 8800M GTX
Toggle full screen
Toggle REF (F3)
Change device (F2)
Display help text
Controls:
Rotate camera
Move camera
Rotate light
Move light
Change light mode (Current: Free)
Change view reference (Current: Camera)
Hidehelp
Quit
Left drag mouse
W,S,A,D,Q,E
Right drag mouse
W,S,A,D,Q,E while holding right mouse
F
V
F1
ESC
View from light's perspectiv
Attach light to car

# Depth Bias



- Полигоны, которые лежат в одной плоскости в 3D-пространстве, могут выглядеть так, как будто это не так, добавив смещение глубины к каждому из них (при вычислении карты глубин)
- Существует два варианта расчета смещения глубины:
  - Если буфер глубины имеет формат UNORM

```
Bias = (float)DepthBias * r + SlopeScaledDepthBias *
MaxDepthSlope;
```

  - где, **r** — минимальное представляемое значение > 0 в формате буфера глудины, преобразованное в float32
  - **MaxDepthSlope** — это максимум горизонтального и вертикального уклона значения глубины в пикселе

```
CD3D11_RASTERIZER_DESC rastDesc = {};
rastDesc.CullMode = D3D11_CULL_FRONT;
rastDesc.FillMode = D3D11_FILL_SOLID;
rastDesc.DepthBias      = DepthBias;
rastDesc.DepthBiasClamp = DepthBiasClamp;
rastDesc.SlopeScaledDepthBias = SlopeScaledDepthBias;
```

https://learn.microsoft.com/en-us/windows/win32/direct3d11/d3d10-graphics-programming-guide-output-merger-stage-depth-bias

# Depth Bias

- Если буфер глубины имеет формат Float

```
Bias = (float)DepthBias * 2**(exponent(max z in primitive) - r) + SlopeScaledDepthBias * MaxDepthSlope;
```

  - где, **r** — количество битов мантиссы в представлении с плавающей запятой (исключая скрытый бит); Например, 23 для float32

  - Затем значение смещения ограничивается следующим образом:

```
if(DepthBiasClamp > 0)
    Bias = min(DepthBiasClamp,
Bias)
else if(DepthBiasClamp < 0)
    Bias = max(DepthBiasClamp,
Bias)
```

  - Значение смещения затем используется для вычисления глубины пикселя

```
if ( (DepthBias != 0) || (SlopeScaledDepthBias !=
0) )
    z = z + Bias
```

https://learn.microsoft.com/en-us/windows/win32/direct3d11/d3d10-graphics-programming-guide-output-merger-stage-depth-bias

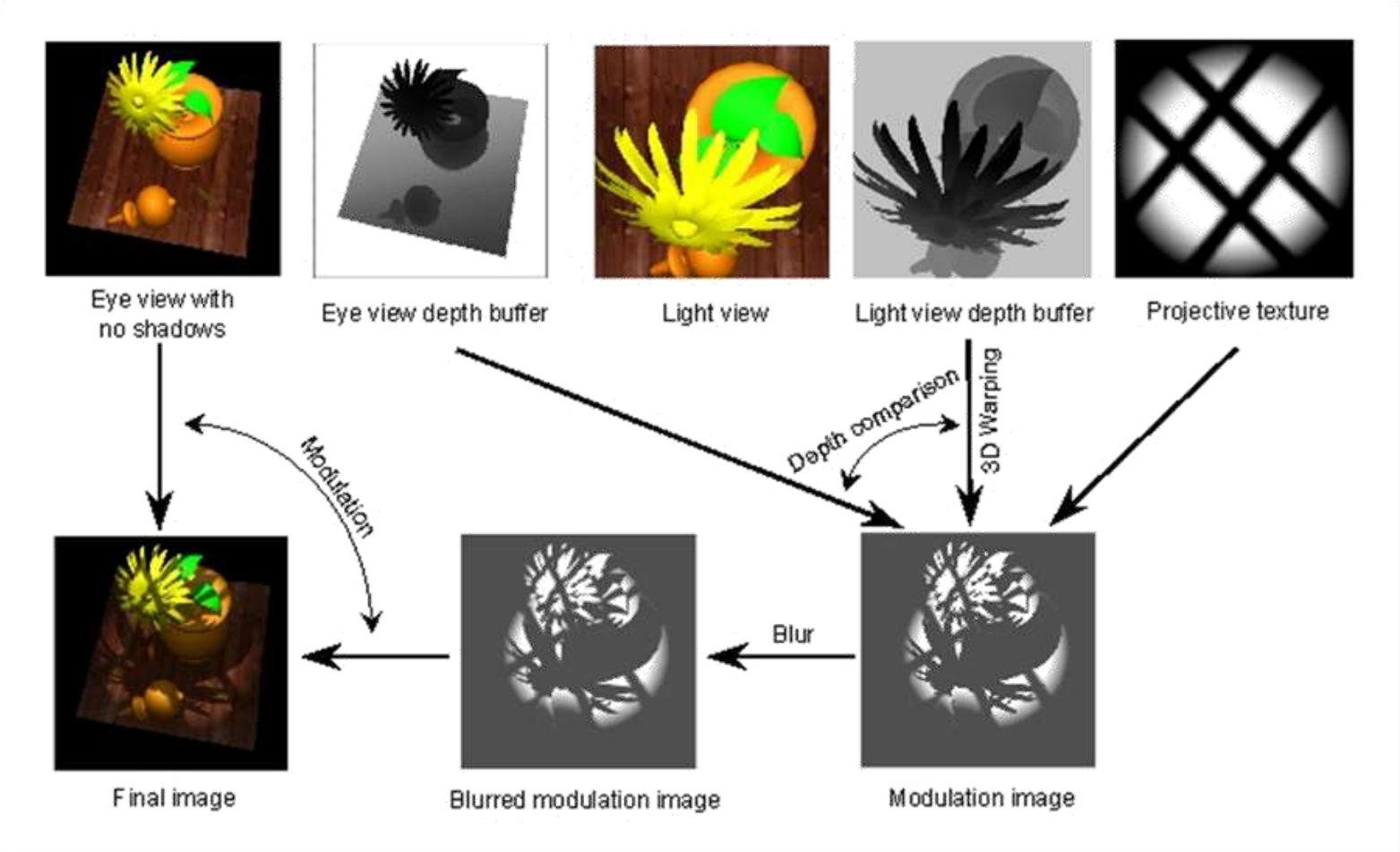
# Depth Bias

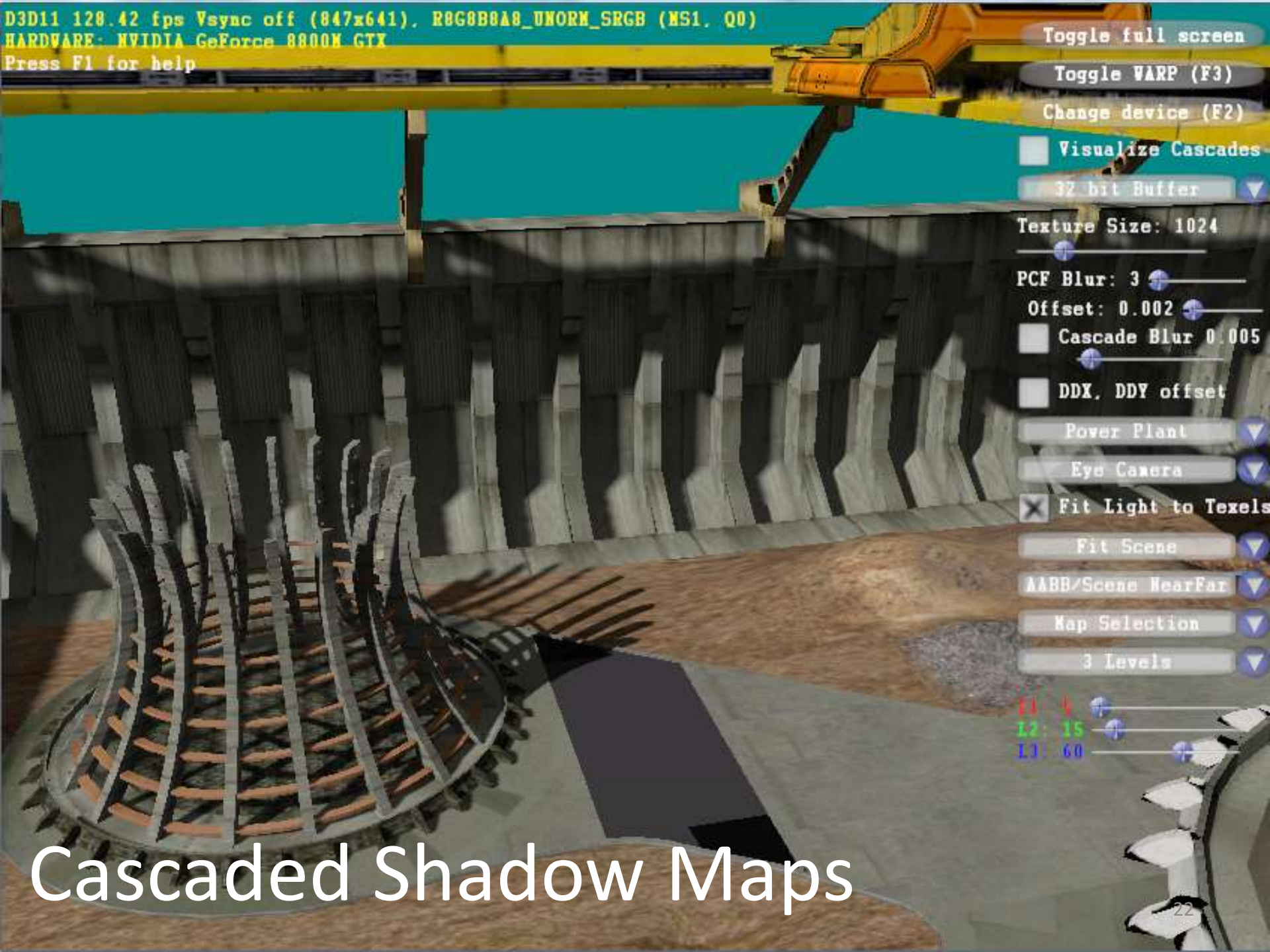
- Операции смещения глубины происходят в вершинах после отсечения, поэтому смещение глубины не влияет на геометрическое отсечение
- Значение смещения является постоянным для данного примитива и прибавляется к значению z для каждой вершины до интерполяции

- Имейте в виду, что использование **DepthBias** и **SlopeScaledDepthBias** может привести к возникновению новых проблем с рендерингом, когда мы рассматриваем полигон, под очень острым углом, уравнение смещения генерирует очень большое значение z
- В итоге, это отталкивает полигон очень далеко от исходной поверхности в карте теней
- Одним из способов облегчить эту проблему является использование **DepthBiasClamp**

https://learn.microsoft.com/en-us/windows/win32/direct3d11/d3d10-graphics-programming-guide-output-merger-stage-depth-bias

# Shadow Maps: Extensions






# Cascaded Shadow Maps

# Main Idea Behind CSM

- Нам не нужно одинаковое количество текселей карты теней для объектов, которые находятся близко и далеко
- Поэтому мы просто используем разные карты теней для объектов, которые находятся близко и далеко
  - И в зависимости от глубины пикселя будем выбирать соответствующую карту теней

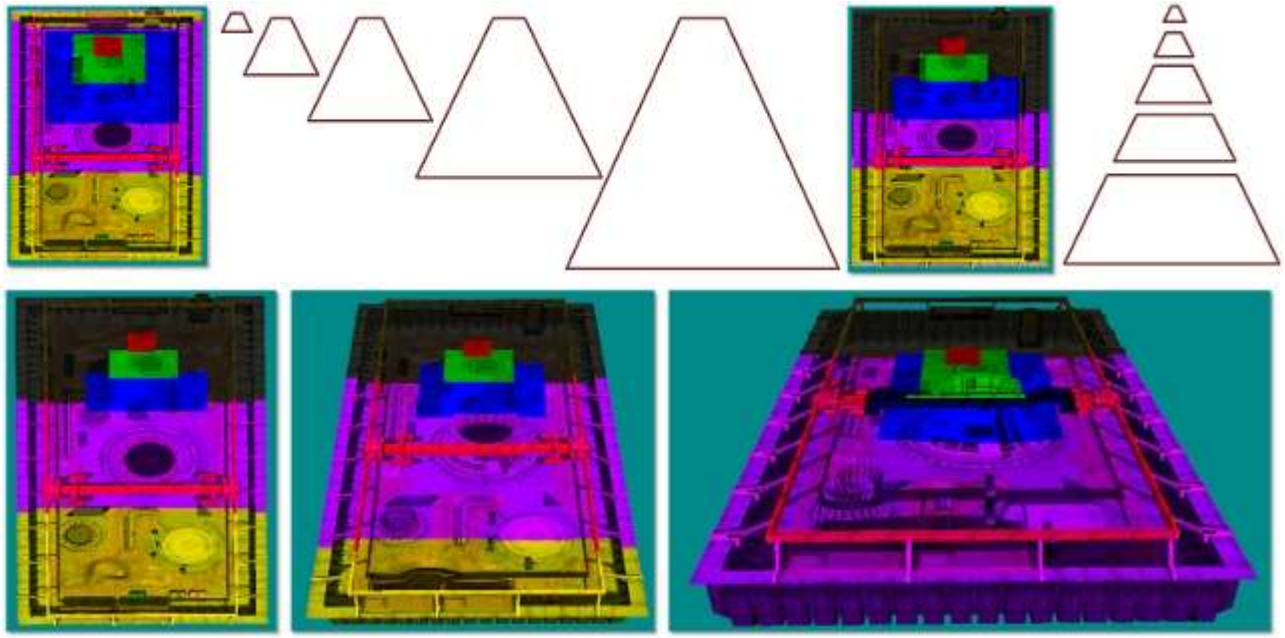
https://learnopengl.com/Guest-Articles/2021/CSM

# CSM Algorithm

- Разделим исходный фрустум камеры на n под-фрустумов
  - где дальняя плоскость i-ого фрустума является ближней плоскостью (i+1)-ого фрустума
- Вычислим плотно подогнанную ортогональную матрицу проекции для каждого фрустума
- Для каждого фрустума визуализируем карту теней, как если бы она была видна из нашего направленного источника света
- Подключим все карты теней к пиксельному шейдеру
- Визуализируем сцену, в соответствии с глубиной пикселя будем выбирать подходящую карту теней

# Cascaded Shadow Maps

- Фрустум камеры делится на несколько сегментов
- Способ его разделения зависит от модификации



https://docs.microsoft.com/en-us/windows/win32/dxtecharts/cascaded-shadow-maps

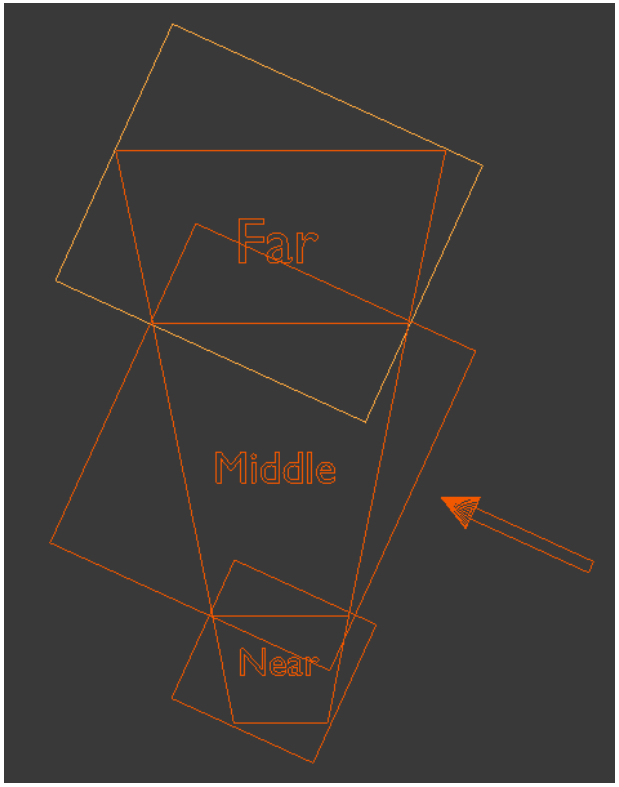
# World Coordinates of the View Frustum

- Нам нужно преобразовать координаты вершин куба NDC в мировое пространство

```
std::vector<Vector4> GetFrustumCornersWorldSpace(const Matrix& view, const Matrix& proj)
{
    const auto viewProj = view * proj;
    const auto inv = viewProj.Invert();

    std::vector<Vector4> frustumCorners;
    frustumCorners.reserve(8);
    for (unsigned int x = 0; x < 2; ++x) {
        for (unsigned int y = 0; y < 2; ++y) {
            for (unsigned int z = 0; z < 2; ++z) {
                const Vector4 pt =
                    Vector4::Transform(Vector4(
                        2.0f * x - 1.0f,
                        2.0f * y - 1.0f,
                        z,
                        1.0f), inv);
                frustumCorners.push_back(pt / pt.w);
            }
        }
    }

    return frustumCorners;
}
```

# Cascades Count

- Фрустум камеры разбивается на каскады
- Количество каскадов зависит от вашего выбора
- Нам нужно рассчитать расстояния для дальней плоскости каждого каскада




```
struct CascadeData
{
    Matrix viewProjMats[CASCADE_COUNT];
    float distances[CASCADE_COUNT];
};
```

# The Light View-Projection Matrix

- Мы знаем направление источника света
- Мы знаем точку в мировом пространстве, на которую он смотрит (для расчёта view матрицы)
  - Центр фрустума

```
Vector3 center = Vector3::Zero;
for (const auto& v : corners) {
    center += v.ToVec3();
}
center /= corners.size();

const auto lightView = Matrix::CreateLookAt(
    center,
    center + inLightDirection,
    Vector3::Up
);
```

# The Light View-Projection Matrix

- Мы знаем размер каскада (для расчёта матрицы проекции)
- Измерение каскада надо проводить во view space

```
float minX = std::numeric_limits<float>::max();
float maxX = std::numeric_limits<float>::lowest();
float minY = std::numeric_limits<float>::max();
float maxY = std::numeric_limits<float>::lowest();
float minZ = std::numeric_limits<float>::max();
float maxZ = std::numeric_limits<float>::lowest();
for (const auto& v : corners) {
    const auto trf = Vector4::Transform(v, lightView);

    minX = std::min(minX, trf.x);
    maxX = std::max(maxX, trf.x);
    minY = std::min(minY, trf.y);
    maxY = std::max(maxY, trf.y);
    minZ = std::min(minZ, trf.z);
    maxZ = std::max(maxZ, trf.z);
}

constexpr float zMult = 10.0f;
minZ = (minZ < 0) ? minZ * zMult : minZ / zMult;
maxZ = (maxZ < 0) ? maxZ / zMult : maxZ * zMult;

auto lightProjection = Matrix::CreateOrthographicOffCenter(minX, maxX, minY, maxY, minZ, maxZ);
```

# Scene Rendering For Each Cascade

- Зная матрицы ViewProj каскадов, нам нужно отрисовать сцену столько раз, сколько у нас каскадов
- Не забудьте использовать правильный viewport

```
struct GS_IN
{
    float4 pos : POSITION;
};

GS_IN VSMain(VS_IN input)
{
    GS_IN output = (GS_IN) 0;

    output.pos = mul(float4(input.pos.xyz, 1.0f), ConstData.World);

    return output;
}
```

```
struct CascadeData
{
    float4x4 ViewProj[4];
    float4 Distances;
};

cbuffer CascBuf : register(b0)
{
    CascadeData CascData;
}

struct GS_OUT
{
    float4 pos : SV_POSITION;
    uint arrInd : SV_RenderTargetArrayIndex;
};

[instance(4)]
[maxvertexcount(3)]
void GSMain(    triangle GS_IN p[3],
                in uint id : SV_GSInstanceID,
                inout TriangleStream<GS_OUT> stream)
{
    [unroll]
    for (int i = 0; i < 3; ++i) {
        GS_OUT gs = (GS_OUT)0;
        gs.pos      = mul(  float4(p[i].pos.xyz, 1.0f),
                            CascData.ViewProj[id]);
        gs.arrInd   = id;
        stream.Append(gs);
    }
}
```

# Finding the Right Layer

- Мы сохраняем расстояния дальних плоскостей в пространстве камеры, поэтому нам нужно получить глубину пикселя также в пространстве камеры
- Поскольку камера смотрит в направлении –Z, мы будем использовать абсолютное значение

```
int layer = 3;
float depthVal = abs(input.ViewPos.z);
for (int i = 0; i < 4; ++i) {
    if (depthVal < CascData.Distances[i]) {
        layer = i;
        break;
    }
}
```

# Making Depth Texture Array

- Width и Height размеры карты теней, а не размеры вашего окна
- В коде HLSL массивы текстур имеют тип `Texture2DArray`

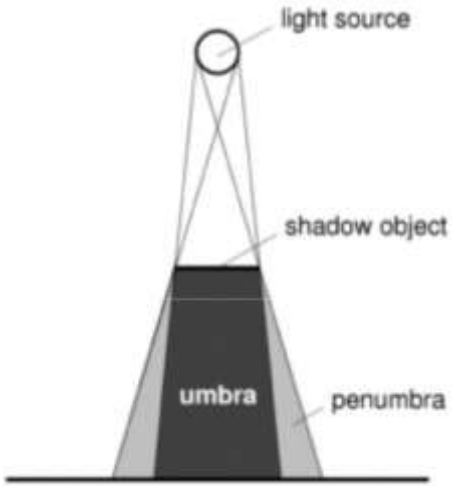
```
D3D11_TEXTURE2D_DESC depthDescription = {};
depthDescription.Width  = width;
depthDescription.Height = height;
depthDescription.ArraySize  = CASCADE_COUNT;
depthDescription.BindFlags  = D3D11_BIND_SHADER_RESOURCE | D3D11_BIND_DEPTH_STENCIL;
depthDescription.Format     = DXGI_FORMAT_R32_TYPELESS,
...

device->CreateTexture2D(&depthDescription, nullptr, &shadowTexArr);

D3D11_DEPTH_STENCIL_VIEW_DESC dViewDesc = {};
dViewDesc.Format = DXGI_FORMAT_D32_FLOAT;
dViewDesc.ViewDimension = D3D11_DSV_DIMENSION_TEXTURE2DARRAY;
dViewDesc.Texture2DArray = {    .MipSlice = 0,
                                .FirstArraySlice = 0,
                                .ArraySize = 4  };
device->CreateDepthStencilView(shadowTexArr, &dViewDesc, &depthDSV);

D3D11_SHADER_RESOURCE_VIEW_DESC srvDesc = {};
srvDesc.Format = DXGI_FORMAT_R32_FLOAT;
srvDesc.ViewDimension = D3D11_SRV_DIMENSION_TEXTURE2DARRAY;
srvDesc.Texture2DArray = {  .MostDetailedMip = 0,
                            .MipLevels = 1,
                            .FirstArraySlice = 0,
                            .ArraySize = 4  };
device->CreateShaderResourceView(shadowTexArr, &srvDesc, &depthSRV);
```

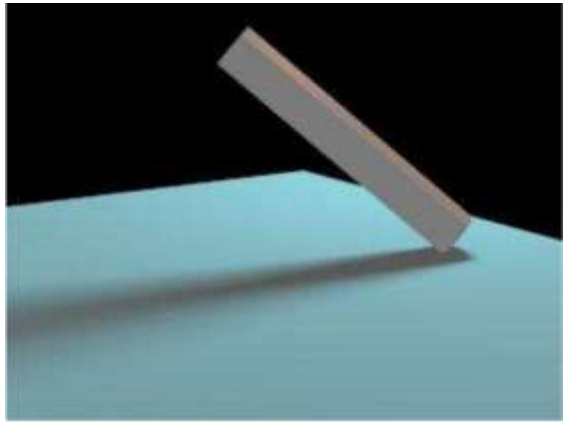
# SOFT SHADOWS

# Soft Shadows

- Возникают из-за «объёмных» источников света
  - Тень (umbra) – полностью скрыта от источника света
  - Полутень (penumbra) – частично скрыта от источника света

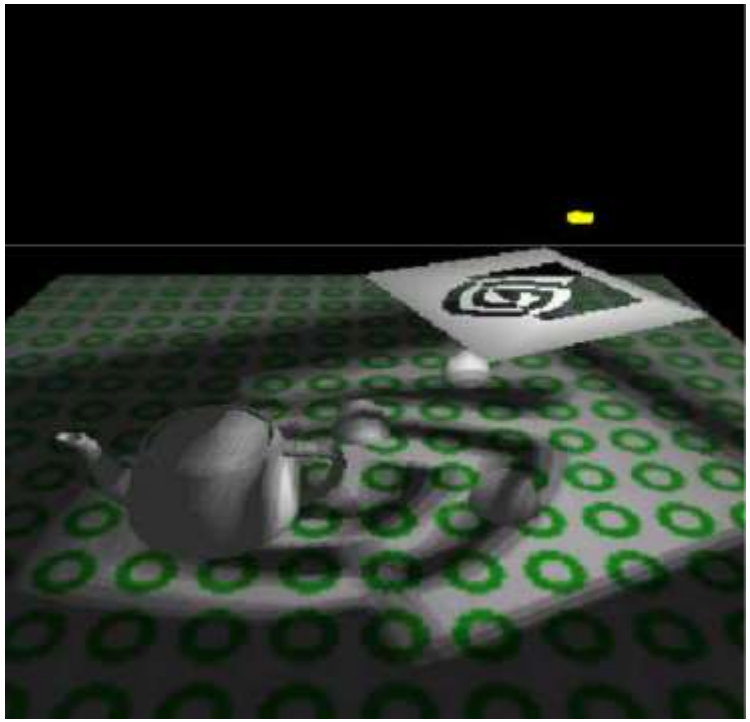
- Можно смоделировать с помощью набора точечных источников света



# Soft shadowing by multiple point light sources

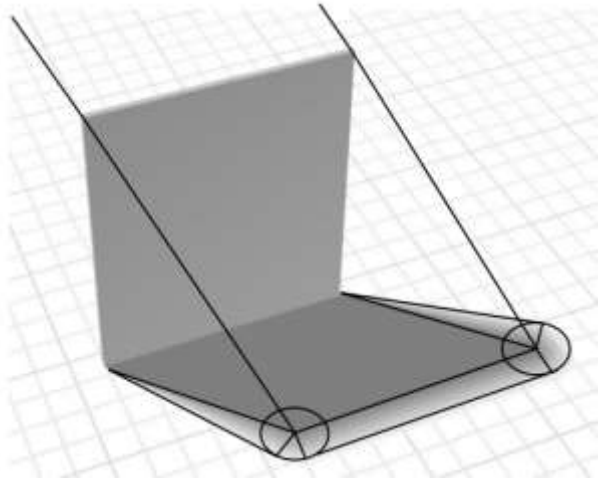
- Аддитивное смешивание используется для накопления вклада каждого источника света
- Мягкость тени зависит от достаточного количества сэмплов
- Время рендеринга сцены линейно увеличивается с количеством сэмплов, используемых для аппроксимации  площади источника света
- Появляются артефакты, если используется недостаточное количество сэмплов



- Недостаток: всё очень медленно

# Other techniques to generate soft shadow : Haines, 2001

- Сначала создайте жесткую тень, а затем нарисуйте рёбра силуэта градиентами, которые идут от темного в центре до белого по краям
  - Градиентные области имеют ширину, пропорциональную высоте ребра силуэта, отбрасывающего тень



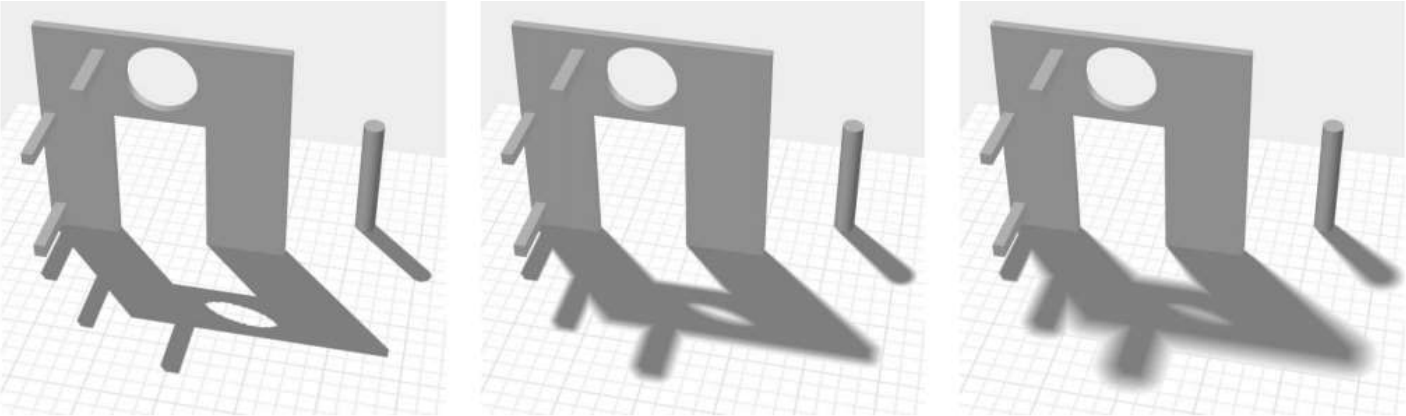


Figure 3: On the left is a hard shadow, the middle shows the effect of a small area light source, the right a larger light source.

# Shadow Maps: Extensions

Принцип, лежащий в основе карт теней, прост, но реализация может быть проблематичной, так как:

- Выбор подходящего значения смещения может быть затруднен
- Трудно удалить артефакты по краям теней
- Работа с маленькими углами обзора

# Shadow Maps: Extensions

**Screen Space Blurred Shadow Mapping**

- Тени отрисовываются в текстуру (в экранном пространстве)
- Затем текстура размывается
- Очень прост в реализации, но может страдать от «затекания» тени и стоимости дополнительных проходов

**Percentage Closer Filtering**

- Результаты сравнения глубины фильтруются
- Т.е. при сравнении глубин также сравниваются глубины вокруг целевого пикселя, с получением усредненного результата

# Comparison Sampler

- Сравнение происходит с первым компонентом текстуры и значением для сравнения
- Для каждого извлеченного текселя (на основе настройки режима фильтрации сэмплера), SampleCmp выполняет сравнение CompareValue из шейдера со значением текселя
  - 1 если сравнение проходит
  - иначе 0
- SampleCmp смешивает результаты 0 и 1 для каждого текселя вместе, как при фильтрации текстур с нормалями (не среднее) и возвращает полученное значение [0..1] в шейдер

```
float Object.SampleCmp(
    SamplerComparisonState S,
    float Location,
    float CompareValue,
    [int Offset]
);
```

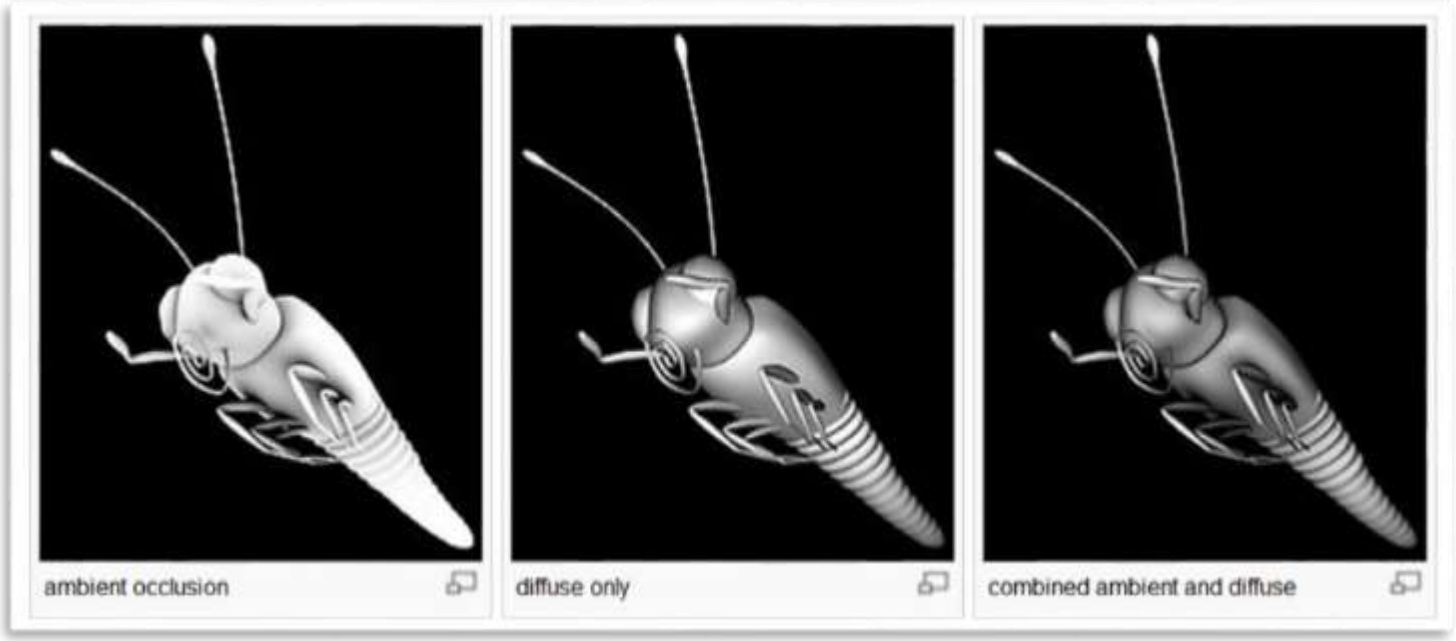
```
depthSamplerDesc.Filter          = D3D11_FILTER_COMPARISON_MIN_MAG_MIP_LINEAR;
depthSamplerDesc.ComparisonFunc  = D3D11_COMPARISON_LESS_EQUAL;
```

# AMBIENT OCCLUSION

# Ambient Occlusion

Окружающее затенение добавляет реализма за счет учета затухания света из-за окклюзии. Оно пытается приблизиться к тому, как свет излучается в реальной жизни

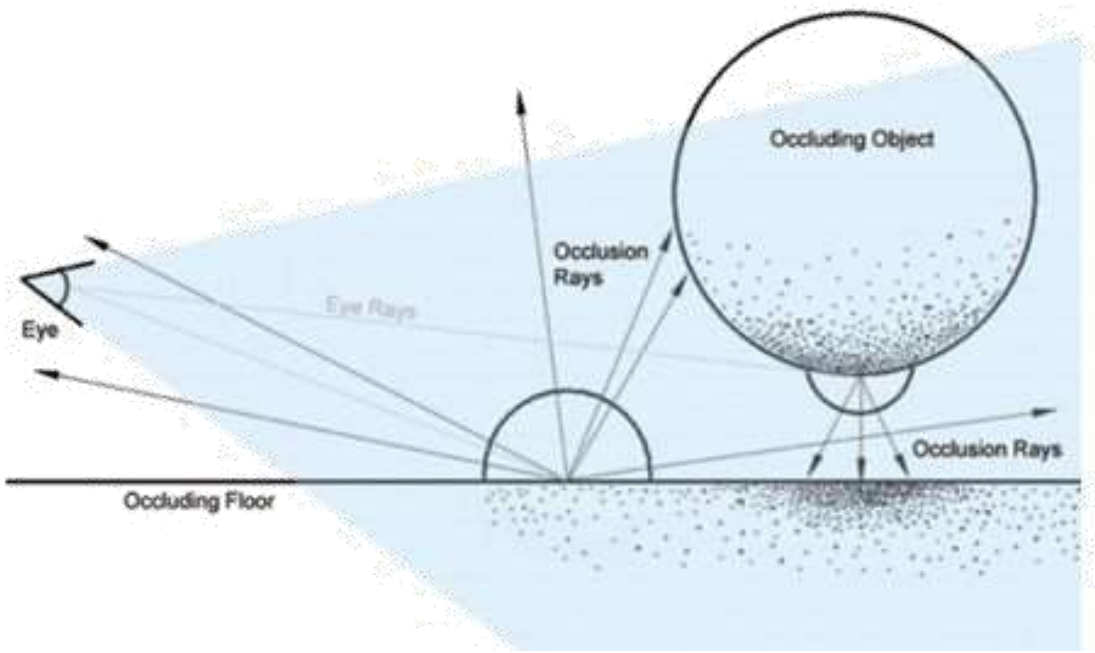
Ambient occlusion — это глобальный метод (в отличие от затенения Фонга, который является локальным методом), это означает, что освещение в каждой точке является функцией другой геометрии в сцене



# Ambient Occlusion

Окружающее затенение чаще всего рассчитывается путем отбрасывания лучей во всех направлениях от поверхности. Лучи, достигающие фона или «неба», увеличивают яркость поверхности, в то время как луч, падающий на любой другой объект, не вносит никакого освещения

В результате точки, окруженные большим количеством геометрии, становятся темными, в то время как точки с небольшой геометрией в видимом полушарии выглядят хорошо освещенными



# Screen Space Ambient Occlusion

SSAO — самая популярная техника (представлена в игре Crysis) , которая способна создать аппроксимацию окружающего затенения за счет использования глубины сцены (буфера глубины)

Он работает путем сравнения текущей глубины пикселя со случайной глубиной вокруг него (берётся несколько сэмплов в случайном направлении), чтобы оценить насколько затенён текущий пиксель. Текущий пиксель затенён, если глубина сэмпла находится дальше, чем значение в буфере глубины

Несмотря на то, что подход очень прост, он также очень эффективен

# Screen Space Ambient Occlusion: Approach

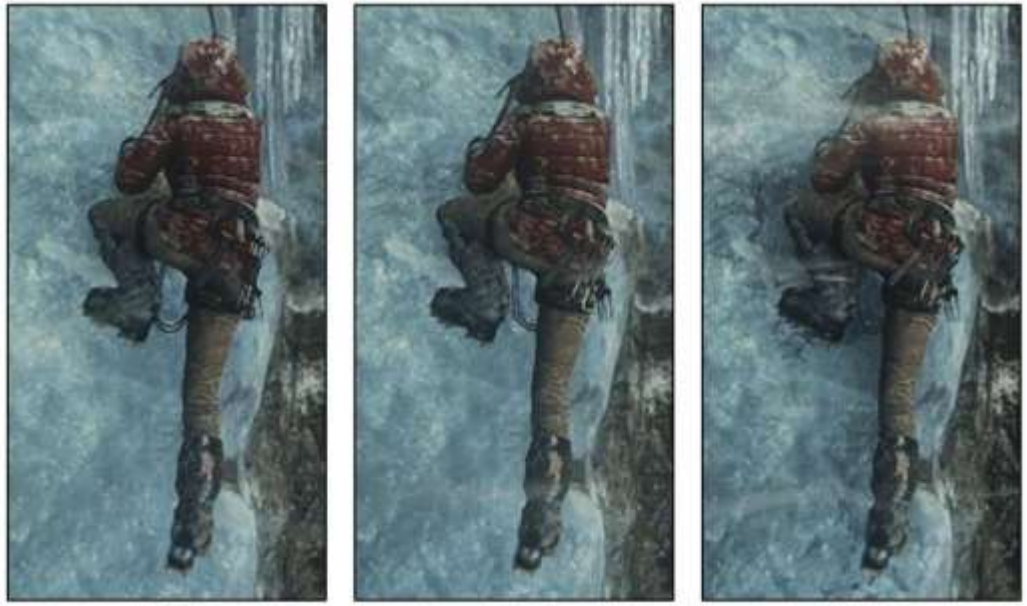
- Для каждого пикселя, шейдер сэмплирует буфер глубины (со смещением на случайный вектор) чтобы оценить затенение
- Выборка выполняется с помощью ядра с произвольным вращением (которое повторяется через каждые N пикселей экрана), гарантирует то, что на выходе будет только высокочастотный шум
- Шум в основном может быть удален с помощью этапа постобработки NxN (сглаживание по Гауссу) с учетом разрывов глубины
- Этот подход позволяет сократить количество выборок глубины примерно до 16 на пиксель (позволяя работать в режиме реального времени) при этом обеспечивая качественный результат

# HBAO & HBAO+

- HBAO — Horizon Based Ambient Occlusion
  - использует физически обоснованный алгоритм, который аппроксимирует интеграл с помощью выборки буфера глубины. По причинам производительности HBAO обычно считается в половинном разрешении, что вызывает мерцание
- HBAO+
  - Используется шахматный рендеринг. Половина пикселей из нового кадра, половина из предыдущего кадра

# VXAO

- VXAO — Voxel Accelerated Ambient Occlusion
  - Три шага: вокселизация сцены, постобработка вокселей, трассировка конусов
  - Не в экранном пространстве, точнее



# FURTHER READING

# Further reading: Shadowing

- Read **Efficient and Robust Shadow Volumes Using Hierarchical Occlusion Culling and Geometry Shaders (GPU Gems 3)**
- Read **DevMaster - Shadow Mapping and Shadow Volumes –** for an excellent introduction to shadowing
- Read **GDC - Shadow Mapping GPU-based Tips and Techniques** – exploring different approaches to shadow mapping

# Further reading: Shadowing

- Read **GPU Gems - Efficient Shadow Volume Rendering – for information on how to render shadow volumes**
- Read **GDC - Advanced Soft Shadow Mapping Techniques** – exploring shadow map softening techniques
- Read **High-Quality Adaptive Soft Shadow Mapping** – for an example of ongoing research
- Read **GPU Gems - Shadow Map Antialiasing** – for information on efficient dithering

# Further reading: Shadowing

- Read **GPU Gems** – **Ambient Occlusion** – providing a good overview of AO
- Read **GPU Gems 2** – **Ambient Occlusion and Indirect Lighting** – exploring forms of indirect lighting
- Read **Hardware Accelerated Ambient Occlusion Techniques on GPUs** – provided an excellent account of GPU screen-space AO

# Homework

- Designers:
  - Добавьте простые карты теней в катамари

- EngineDevs:
  - Добавьте каскадные карты теней в катамари
  - Используйте PCF